

Architecture: Addressing Modes

COMS10015

Dr Tom Deakin

University of Bristol

Week 14

These slides are based on material from Dan Page.

Addressing modes

- ▶ Previously saw that operands can be *immediate* values, registers, memory addresses.
- ▶ How the operands are interpreted is the *addressing mode*.
- ▶ For example, the operand x might be a register address or a memory address.
- ▶ Note: quite typical for instructions to have 1 output and n inputs.
- ▶ E.g., 2 inputs and 1 output for binary operations like addition.

Notation

- ▶ We'll use $GPR[i]$ to refer to the i^{th} general-purpose register.
- ▶ We'll use $MEM[i]$ to refer to the i^{th} location in memory (with address i).

Immediate mode

Example

In C, we might write `r = t + 1;`.

How do we represent 1 as an operand to the addition instruction?

We can use an *immediate*: the operand *IS* the data.

`add r0, r1, #1` $\equiv GPR[0] \leftarrow GPR[1] + 1$

Direct mode

Example

In C, we might write `r = t + 1;`.

How do we represent `t` as an operand to the addition instruction?

We can use an *direct*: the operand is the *ADDRESS* of the data.

Could be register direct, i.e., $GPR[1]$ contains the value of `t`.

`add r0, r1, #1` $\equiv GPR[0] \leftarrow GPR[1] + 1$.

Or memory direct, i.e., `t` is stored in memory, and the instruction set is a memory-memory or register-memory architecture.

What are variables (in programming languages like C)?

Direct addressing mode example: Armv7

```
mov r0, #0          @ r0 = 0
mov r1, #1          @ r1 = 1
loop:               @ this is a label (not an instruction)
add r2, r0, r1      @ r2 = r0 + r1
mov r0, r1          @ shuffle over values
mov r1, r2
cmp r1, #200        @ branch if r1 < 200 (two instructions)
blt loop
```

Only uses:

- ▶ immediate values (in Arm this is constants with # symbol).
- ▶ direct addressing (in Arm this is the names of registers).

Thanks ChatGPT for this example of Fibonacci using only registers.

Indirect mode

Example

In C, we might write `r = *t + 1;`.

Note that `t` is now a pointer (e.g., some array element).

How do we access `t` as an operand to the addition instruction?

Recall from C that the `t` pointer variable has the value of an address in memory. The `t` itself is stored somewhere too.

We could first load the value of `t` from memory into a register, and use a direct mode addition instruction (2 instructions).

Indirect mode

Example

In C, we might write `r = *t + 1;`.

Note that `t` is now a pointer (e.g., some array element).

How do we access `t` as an operand to the addition instruction?

Or, we can use an *indirect*: the operand is the *ADDRESS of the ADDRESS* in memory of the data.

`add r0, [r1], #1` $\equiv GPR[0] \leftarrow MEM[GPR[1]] + 1$.

I.e., `GPR[1]` contains the address in memory of where `t` is stored.

(Could also do memory indirect: `MEM[MEM[1]]`)

Indirect addressing mode example: Armv7

```
ldr r2, #0x20000000    @ r2 = &fib_a
mov r1, #0
str r1, [r2]           @ *fib_a = 0
ldr r3, #0x20000004    @ r3 = &fib_b
mov r1, #1
str r1, [r3]           @ *fib_b = 1
loop:
ldr r4, [r2]           @ r4 = *fib_a
ldr r5, [r3]           @ r5 = *fib_b
add r6, r4, r5         @ compute next number
str r5, [r2]           @ store results: *fib_a = r5 (r5 is *fib_b)
str r6, [r3]           @ *fib_b = r6
cmp r6, #200           @ branch if r6 < 200 (two instructions)
blt loop
```

Indirect addressing in Arm with [] notation. Register contains a memory address.

Encoding pressure

We only have so many bits in the instruction for each opcode and operand. The challenge of fitting all the instructions in causes *encoding pressure*. Recall, for l bits, we can represent numbers in the range:

unsigned $0 \leq x \leq 2^l - 1$

signed $-2^{l-1} \leq x \leq 2^{l-1} - 1$

Recall: the mapping of those numbers to the instructions (opcode and operands) is the instruction encoding.

For operands: we can use the bits to represent an immediate value in that range, or the address in the process state (register or memory).

Addressing Modes and Encoding Pressure

Example: I -bits for an operand

For unsigned, $0 \leq x \leq 2^I - 1$ with $I = 8$, gives us 256 integers.

If only have immediate addressing mode, limited to 256 operand values: immediates up to 256, 256 locations in memory/registers.

With **direct** addressing mode can access 256 *word-size* parts of memory/registers. Allows us to compute on word-sized numbers: i.e., use the values stored *inside* the register/memory.

With **indirect** addressing mode, can access word-size addresses of memory. Allows us to use the memory addresses *inside* registers. Not as limited by operand encoding: doesn't require word-size operands.

Note too that direct and indirect allow the instruction to operate on non-constant (dynamic) data.

Indexed addressing mode

Example

```
typedef struct __pt {  
    uint8_t r;  
    uint8_t g;  
    uint8_t b;  
} pt;
```

```
pt p;
```

How do we access `p.g`?

Could do this manually:

1. load `&p` into a register: $GPR[0] = \&p$
2. add offset to `g`: $GPR[1] = GPR[0] + \text{sizeof}(\text{uint8_t})$
3. load `p.g` into register: $GPR[2] = \text{MEM}[GPR[1]]$

Indexed addressing mode

Use *indexed* addressing mode: the operand gives the *ADDRESS plus OFFSET* to get the address of the data:

- ▶ `ldrb r2, [r0, #1]` $\equiv GPR[2] \leftarrow MEM[GPR[0] + 1]$
- ▶ `ldrb r2, [r0, r1]` $\equiv GPR[2] \leftarrow MEM[GPR[0] + GPR[1]]$

Examples use the Armv7 ISA. Similar notation to indirect addressing but address is computed.

Auto-indexed addressing mode

Example

```
for (int i = 0; i < N; ++i) {  
    r += A[i];  
}
```

How do access $A[i]$ and then update t .

Could do this manually:

1. $GPR[0] \leftarrow \&A$
2. Add offset: $GPR[1] \leftarrow GPR[0] + GPR[3]$
3. Load $A[i]$: $GPR[2] \leftarrow MEM[GPR[1]]$
4. Increment offset: $GPR[3] = GPR[3] + 1$

Auto-indexed addressing mode

Use *Auto-indexed* addressing mode: use indexed addressing mode, and update the address/offset.

`ldrb r2, [r1], #1` $\equiv GPR[2] \leftarrow MEM[GPR[1]]$ then $GPR[1] \leftarrow GPR[1] + 1$

Arm calls this post-indexed addressing.

Taking stock

- ▶ To access data we need to compute an address, and then access it.
- ▶ In high-level programming languages (e.g., C), addressing modes are often hidden by language syntax.
- ▶ ISA needs to provide them so convenient to implement those languages and write programs.
- ▶ CISC-like ISAs typically give more options for addressing modes than RISC-like.

Further reading

- ▶ Wikipedia: Addressing mode.
https://en.wikipedia.org/wiki/Addressing_mode.
- ▶ D. Page. Section 5.3: Addressing modes. In: A Practical Introduction to Computer Architecture. 1st ed. Springer, 2009.
- ▶ A.S. Tanenbaum and T. Austin. Section 5.3: Addressing. In: Structured Computer Organisation. 6th ed. Prentice Hall, 2012.
- ▶ W. Stallings. Chapter 13: Instruction sets: addressing modes and formats. In: Computer Organisation and Architecture. 9th ed. Prentice Hall, 2013.